# Lab Exercise 8- Create POD in Kubernetes

## Objective:

- Understand the basic structure and syntax of a Kubernetes Pod definition file (YAML).
- Learn to create, inspect, and delete a Pod in a Kubernetes cluster.

## Prerequisites

- Kubernetes Cluster: You need a running Kubernetes cluster. You can set up a local cluster using tools like Minikube or kind, or use a cloud-based Kubernetes service.
- kubectl: Install and configure kubectl to interact with your Kubernetes cluster.
- Basic Knowledge of YAML: Familiarity with YAML format will be helpful as Kubernetes resource definitions are written in YAML.

## Step-by-Step Guide

### Step 1: Create a YAML File for the Pod

We'll create a Pod configuration file named **pod-example.yaml**

```
apiVersion: v1
kind: Pod
metadata:
  name: my-pod
  labels:
    app: web
spec:
  containers:
    - name: my-container
      image: nginx:latest
```

### Explanation of the YAML File

- apiVersion: Specifies the version of the Kubernetes API to use. For Pods, it's typically v1.
- kind: The type of object being created. Here it's a Pod.
- metadata: Provides metadata about the object, including name and labels. The name must be unique within the namespace, and labels help in identifying and organizing Pods.
- spec: Contains the specifications of the Pod, including:
    - containers: Lists all containers that will run inside the Pod. Each container needs:
        - name: A unique name within the Pod.
        - image: The Docker image to use for the container.
        - ports: The ports that this container exposes.
        - env: Environment variables passed to the container.

### Step 2: Apply the YAML File to Create the Pod

Use the kubectl apply command to create the Pod based on the YAML configuration file.

```
kubectl apply -f pod-example.yaml
```

This command tells Kubernetes to create a Pod as specified in the pod-example.yaml file.

```
[mohdanas@Mohds-MacBook-Air Desktop % touch pod-example.yaml
[mohdanas@Mohds-MacBook-Air Desktop % rm pod-example.yaml
[mohdanas@Mohds-MacBook-Air Desktop % vim pod-example.yaml
 mohdanas@Mohds-MacBook-Air Desktop % kubectl apply -f pod-example.yaml

 pod/my-pod created
 mohdanas@Mohds-MacBook-Air Desktop %
```

### Step 3: Verify the Pod Creation

To check the status of the Pod and ensure it's running, use:

```
kubectl get pods
```

```
mohdanas@Mohds-MacBook-Air Desktop % kubectl get pods

NAME      READY   STATUS    RESTARTS   AGE
my-pod    1/1     Running   0          22s
mohdanas@Mohds-MacBook-Air Desktop %
```

This command lists all the Pods in the current namespace, showing their status, restart count, and other details.

You can get detailed information about the Pod using:

```
kubectl describe pod my-pod
```

```
mohdanas@Mohds-MacBook-Air Desktop % kubectl describe pod my-pod

Name:             my-pod
Namespace:        default
Priority:         0
Service Account:  default
Node:             minikube/192.168.49.2
Start Time:       Fri, 27 Feb 2026 22:55:29 +0530
Labels:           app=web
Annotations:      <none>
Status:           Running
IP:               10.244.0.5
IPs:
  IP:  10.244.0.5
Containers:
  my-container:
    Container ID:   docker://c15c4f0a41d2ceb95825680070647ab49a7b530e30dc10081e5c6e0ca862b5b4
    Image:          nginx:latest
    Image ID:       docker-pullable://nginx@sha256:0236ee02dcbce00b9bd83e0f5fbc51069e7e1161bd59d99885b
3ae1734f3392e
    Port:           <none>
    Host Port:      <none>
    State:          Running
      Started:      Fri, 27 Feb 2026 22:55:45 +0530
    Ready:          True
    Restart Count:  0
    Environment:    <none>
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-fz47c (ro)
Conditions:
  Type                        Status
  PodReadyToStartContainers   True
  Initialized                 True
  Ready                       True
  ContainersReady             True
  PodScheduled                True
Volumes:
  kube-api-access-fz47c:
    Type:                    Projected (a volume that contains injected data from multiple sources)
    TokenExpirationSeconds:  3607
    ConfigMapName:           kube-root-ca.crt
    ConfigMapOptional:       <nil>
    DownwardAPI:             true
QoS Class:                   BestEffort
Node-Selectors:              <none>
Tolerations:                 node.kubernetes.io/not-ready:NoExecute op=Exists for 300s
                             node.kubernetes.io/unreachable:NoExecute op=Exists for 300s
Events:
  Type    Reason     Age   From               Message
  ----    ------     ----  ----               -------
  Normal  Scheduled  43s   default-scheduler  Successfully assigned default/my-pod to minikube
  Normal  Pulling    43s   kubelet            Pulling image "nginx:latest"
  Normal  Pulled     27s   kubelet            Successfully pulled image "nginx:latest" in 15.466s (15.
466s including waiting). Image size: 180544671 bytes.
  Normal  Created    27s   kubelet            Container created
  Normal  Started    27s   kubelet            Container started
mohdanas@Mohds-MacBook-Air Desktop %
```

This command provides detailed information about the Pod, including its events, container specifications, and resource usage.

### Step 4: Interact with the Pod

You can interact with the running Pod in various ways, such as accessing the logs or executing commands inside the container.

**View Logs: To view the logs of the container in the Pod:**

```
kubectl logs my-pod
```

```
mohdanas@Mohds-MacBook-Air Desktop % kubectl logs my-pod
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/02/27 17:25:45 [notice] 1#1: using the "epoll" event method
2026/02/27 17:25:45 [notice] 1#1: nginx/1.29.5
2026/02/27 17:25:45 [notice] 1#1: built by gcc 14.2.0 (Debian 14.2.0-19)
2026/02/27 17:25:45 [notice] 1#1: OS: Linux 6.10.14-linuxkit
2026/02/27 17:25:45 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
2026/02/27 17:25:45 [notice] 1#1: start worker processes
2026/02/27 17:25:45 [notice] 1#1: start worker process 29
2026/02/27 17:25:45 [notice] 1#1: start worker process 30
2026/02/27 17:25:45 [notice] 1#1: start worker process 31
2026/02/27 17:25:45 [notice] 1#1: start worker process 32
2026/02/27 17:25:45 [notice] 1#1: start worker process 33
2026/02/27 17:25:45 [notice] 1#1: start worker process 34
2026/02/27 17:25:45 [notice] 1#1: start worker process 35
2026/02/27 17:25:45 [notice] 1#1: start worker process 36
mohdanas@Mohds-MacBook-Air Desktop %
```

**Execute a Command: To run a command inside the container:**

```
kubectl exec -it my-pod -- /bin/bash
```

The -it flag opens an interactive terminal session inside the container, allowing you to run commands.

```
mohdanas@Mohds-MacBook-Air Desktop % kubectl exec -it my-pod -- /bin/bash

root@my-pod:/#
```

### Step 5: Delete the Pod

To clean up and remove the Pod when you're done, use the following command:

```
kubectl delete pod my-pod
```

This command deletes the specified Pod from the cluster.

```
mohdanas@Mohds-MacBook-Air Desktop % kubectl delete pod my-pod

pod "my-pod" deleted
mohdanas@Mohds-MacBook-Air Desktop %
```